Practical 12

Part A:

Aim:

1. Word frequency: split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator.

Code:

WordFrequency.java

```
import java.util.*;

public class WordFrequency {
    public static void main(String[] args) {
        String paragraph = "Java is easy and Java is powerful.
"
                + "Java is popular and easy to learn.";

        // Normalize case and remove punctuation
        String cleaned = paragraph.toLowerCase()
                .replaceAll("[^a-z0-9\\s]", "");

        String[] words = cleaned.trim().split("\\s+");

        // Count word frequencies
        Map<String, Integer> frequency = new HashMap<>();

        for (String word : words) {
            frequency.put(word, frequency.getOrDefault(word, 0)
+ 1);
        }
```

```
        // Sort entries by frequency descending, then word
ascending
        List<Map.Entry<String, Integer>> entries =
                new ArrayList<>(frequency.entrySet());

        entries.sort(
                Comparator
                        .<Map.Entry<String,
Integer>>comparingInt(
                                Map.Entry::getValue)
                        .reversed()
                        .thenComparing(Map.Entry::getKey)
        );

        System.out.println("Word Frequencies (Highest
First):");
        for (Map.Entry<String, Integer> entry : entries) {
            System.out.println(entry.getKey() + " : " +
entry.getValue());
        }
    }
}
```

Output:



2. Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.

Code:

UniqueVisitors.java

```
import java.util.*;

public class UniqueVisitors {
    public static void main(String[] args) {
        List<String> logins = Arrays.asList(
                "Amit", "Riya", "Karan", "Amit",
                "Sara", "Riya", "Neha", "Karan"
        );

        // HashSet stores unique users
        Set<String> uniqueUsers = new HashSet<>(logins);

        // LinkedHashSet stores unique users in first-seen
order
        Set<String> orderedUsers = new LinkedHashSet<>(logins);

        System.out.println("Unique visitor count: " +
uniqueUsers.size());

        System.out.println("First three distinct visitors:");
        int count = 0;

        for (String user : orderedUsers) {
            if (count == 3) {
                break;
            }

            System.out.println(user);
```

```
                count++;
            }
        }
}
```

Output:



3. (For Advanced Learners – home) Build a leaderboard from player scores held in a Map and print the top-N sorted by score (ties broken by name); choose the data and sorting details.

Code:

LeaderBoard.java

```
import java.util.*;

public class Leaderboard {
    public static void main(String[] args) {
        Map<String, Integer> scores = new HashMap<>();

        scores.put("Zara", 950);
        scores.put("Amit", 870);
        scores.put("Riya", 950);
        scores.put("Karan", 720);
        scores.put("Neha", 870);
        scores.put("Sara", 990);

        int topN = 4;

        List<Map.Entry<String, Integer>> leaderboard =
                new ArrayList<>(scores.entrySet());

        leaderboard.sort(
                Comparator
                        .<Map.Entry<String,
Integer>>comparingInt(
                                Map.Entry::getValue)
                        .reversed()
                        .thenComparing(Map.Entry::getKey)
        );

        System.out.println("Top " + topN + " Players:");
```

```
        for (int i = 0; i < Math.min(topN, leaderboard.size());
i++) {
            Map.Entry<String, Integer> player =
leaderboard.get(i);

            System.out.println(
                    (i + 1) + ". " + player.getKey()
                            + " - " + player.getValue()
            );
        }
    }
}
```

Output:



Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is the difference between a List, a Set and a Map?

Ans.

Feature	Lists	Set	Map
Stores	Individual elements	Unique elements	Key-value pairs
Duplicates	Allowed	Not Allowed	Keys must be unique; values can repeat
Order	Depends on implementation; usually preserves insertion order	Depends on implementation	Depends on implementation
Access	By index	Usually by iteration or lookup	By key
Example	`ArrayList`, `LinkedList`	`HashSet`, `TreeSet`	`HashMap`, `TreeMap`

2. When do you use a HashMap versus a TreeMap?

Ans.

Use `HashMap` when fast key-based access is the priority and key ordering is unnecessary. Use `TreeMap` when you need keys sorted or operations such as finding the smallest key or retrieving a range of keys.

Note: A `TreeMap` can support null keys only if its comparator explicitly permits them. Its default natural ordering does not.

3. What is the difference between Comparable and Comparator?

Ans.

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo()	compare()
Purpose	Defines natural ordering	Defines custom ordering
Implementation	Usually inside the class being sorted	Usually external to that class
Number of orderings	Generally one natural ordering	Multiple custom orderings are possible